

BOOTCAMP MLOps

BOOTCAMP MLOps:

What we will learn to day:
Introduction to CI with Automated Testing

Danh Pham

Goals

- Know basic concepts of CI and Github Actions.
- Know how to write test for python code.
- Create your first Github Action pipeline and run it.

Contents

What is Continuous Integration?

GitHub Actions Core Concepts

Automated Testing with pytest

Hands-on Lab

Building Our First CI Pipeline

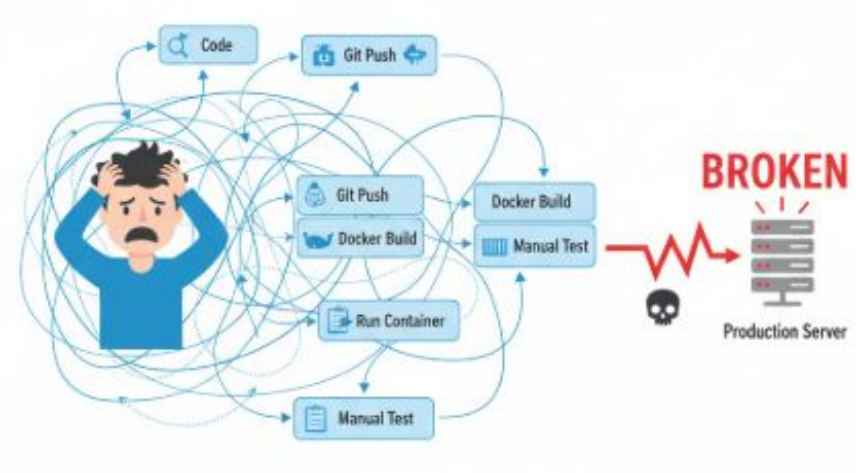
What is Continuous Integration (CI)?

The "Pain" We're Solving

The Manual, Risky Workflow

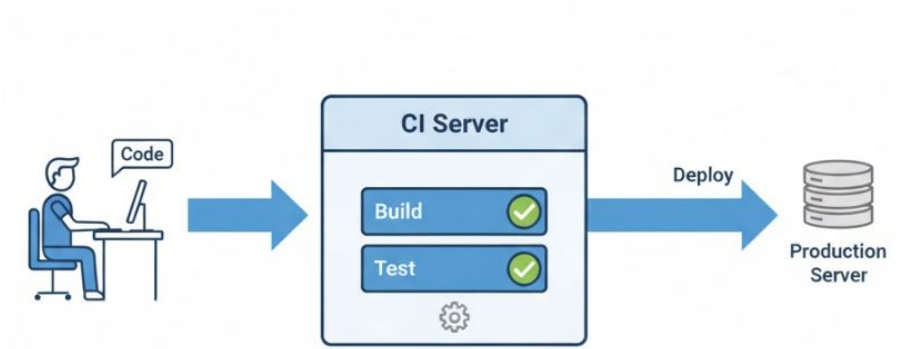
- Write new code, then git push.
- Manually run docker build and the container.
- Manually test the API (e.g., in /docs).

This process is slow, error-prone, and doesn't scale. Forgetting a step or pushing a bug breaks production.



The Solution: Continuous Integration (CI)

- **What is Continuous Integration (CI)?**
- "The practice of merging all developers' working copies to a shared mainline several times a day."
- **In simple terms:** Automatically **building** and **testing** your code every time a change is pushed.



Why Bother with CI?

CI offers Confidence, Consistency, and Collaboration to your team.

Confidence:

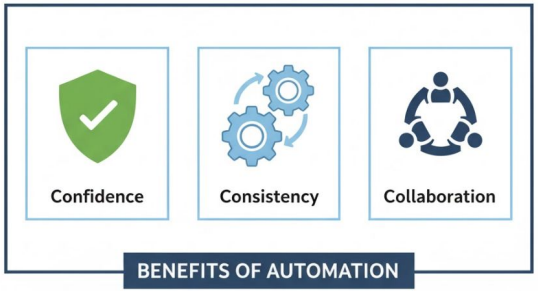
- Bugs are immediately caught, not weeks later.
- Refactor code bravely with the "safety net" to catch errors.

Consistency:

- The same tests run in the same environment, every time.
- Eliminates "But it works on my machine!" issues.

Collaboration:

- The whole team sees the build status.
- A "failing build" publicly signals an immediate fix is needed.

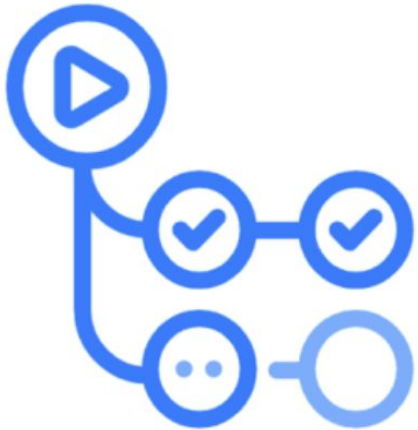


COLE

GitHub Actions Core Concepts

Our Tool: GitHub Actions

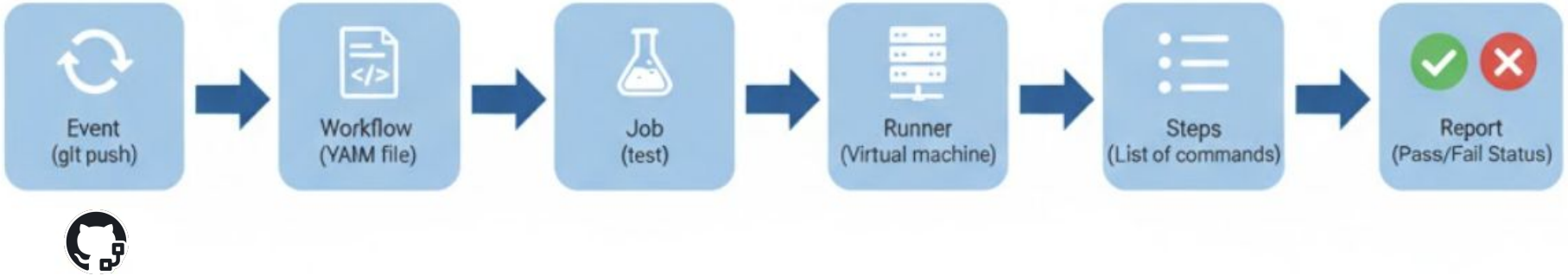
- GitHub Actions is an automation engine built right into GitHub.
- It's "event-driven." It listens for events (like git push) and reacts by running a job.
- It's configured with simple YAML files you commit to your repository.
- It's free for public repositories!



GitHub Actions

COLE

How GHA Works: The Big Picture



A git push happens

GitHub sees a YAML file in .github/workflows/ that "listens" for that event

The workflow starts a job (e.g., a "test" job)

The job runs on a fresh virtual machine called a runner

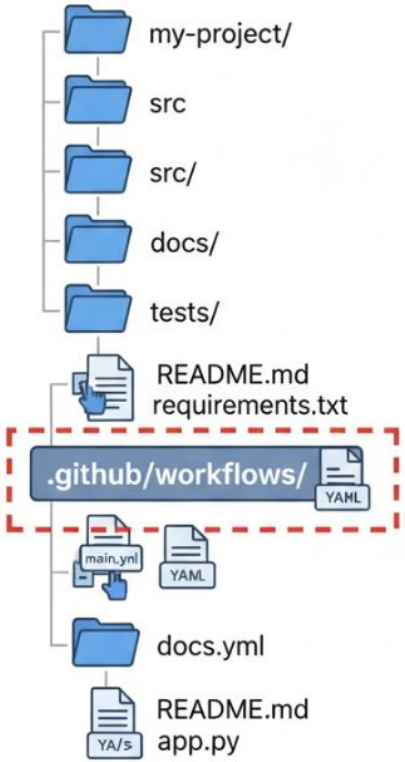
The runner follows the steps you defined (e.g., install code, run tests)

The runner reports a **Pass** or **Fail** status back to GitHub.

Core Concept 1: Workflows

- A Workflow is the top-level automation.
- It's just a YAML file (e.g., main.yml).
- It lives in a special directory: .github/workflows/ in your project root.
- You can have multiple workflow files (e.g., one for testing, one for documentation).
- # .github/workflows/main.yml

name: My First CI Pipeline



Core Concept 2: Events

- An **Event** is the trigger that starts a workflow.
- You define it with the on key.
- The most common event is push.

```yaml
name: CI Pipeline

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

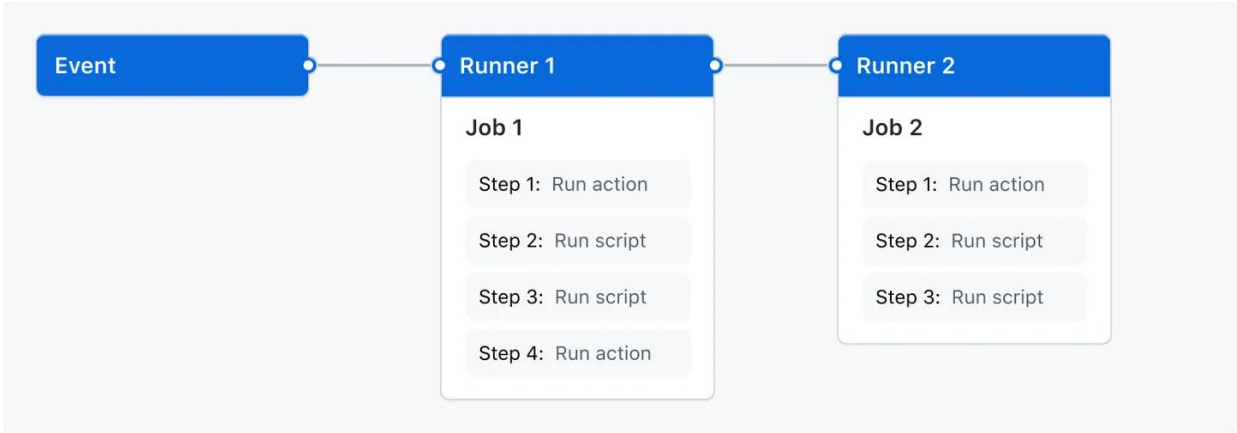
      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test
```

COLE

https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#push

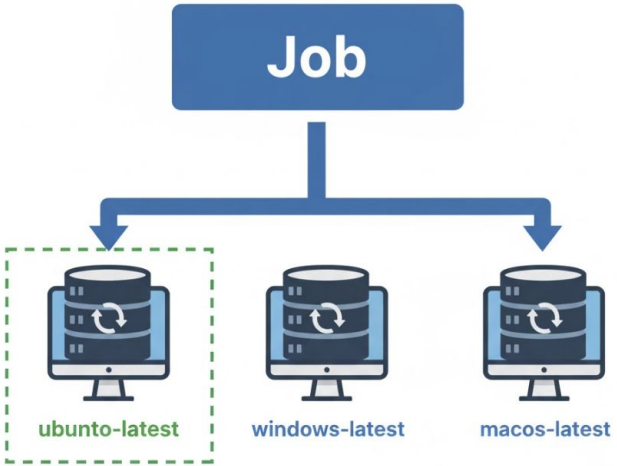
Core Concept 3: Jobs



- A Job is a set of steps that execute on a runner.
- Workflows are made of one or more jobs.
- Jobs run in parallel by default (but can be made sequential).

Core Concept 4: Runners

- A **Runner** is the virtual server that runs your job.
- GitHub provides hosted runners for Linux, Windows, and macOS.
- You define it with runs-on:.
- We'll always use ubuntu-latest as it's the fastest and most common.
- Keyword: `runs-on`

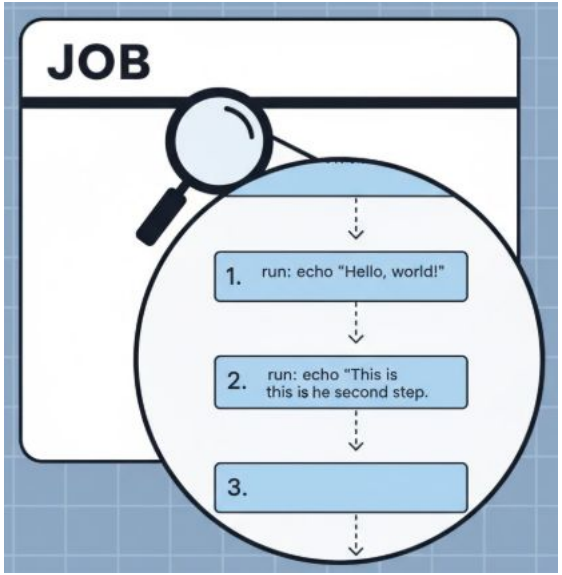


https://docs.github.com/en/actions/concepts/runners/github-hosted-runners

Core Concept 5: Steps

- **Steps** are the individual commands that make up a job.
- Steps run in order.
- A step can be a shell command (run:) or a pre-built script (uses:).
- This is the "meat" of your workflow.

```
# .github/workflows/main.yml
  ...
  jobs:
    test:
      runs-on: ubuntu-latest
      steps:
        - run: echo "Hello, world!"
        - run: echo "This is the second step."
```



https://docs.github.com/en/actions/concepts/runners/github-hosted-runners

Core Concept 6: Actions

- An **Action** (the uses: keyword) is a pre-built, reusable command.
- Why write complex scripts if someone already did?
    - actions/checkout@v4: A mandatory action that *checks out your repository's code* onto the runner.
    - actions/setup-python@v5: An action that *installs Python* for you.




```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with:
      python-version: '3.10'
```

Anatomy of a Workflow File

```yaml
name: learn-github-actions
run-name: ${{ github.actor }} is learning GitHub Actions
on: [push]
jobs:
  check-bats-version:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm install -g bats
      - run: bats -v
```

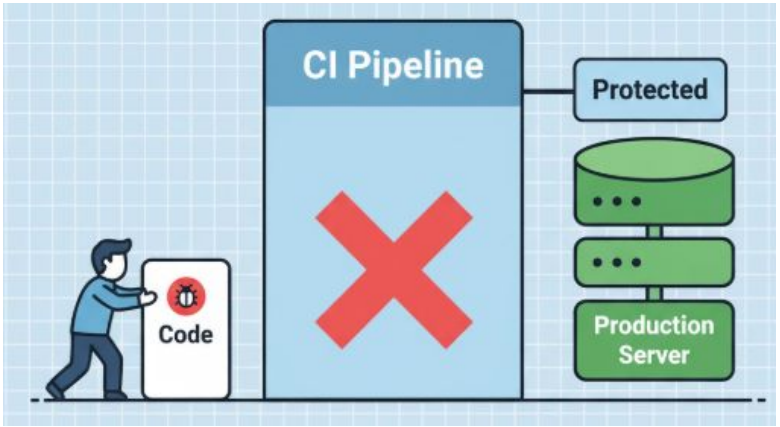
COLE

https://docs.github.com/en/actions/tutorials/create-an-example-workflow

Automated Testing with pytest

The "Safety Net"

- **"CI without testing is just Continuous Integration of Bugs."**
- The *real* power of CI is its role as a **gatekeeper**.
- The pipeline's most important job is to answer one question:
- "Does this new code break anything?"
- If tests fail, the pipeline fails, and the code is blocked.

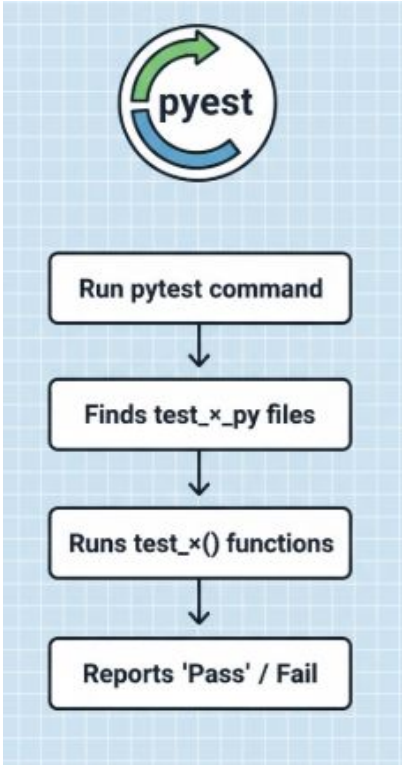


COLE

Our Testing Tool: pytest

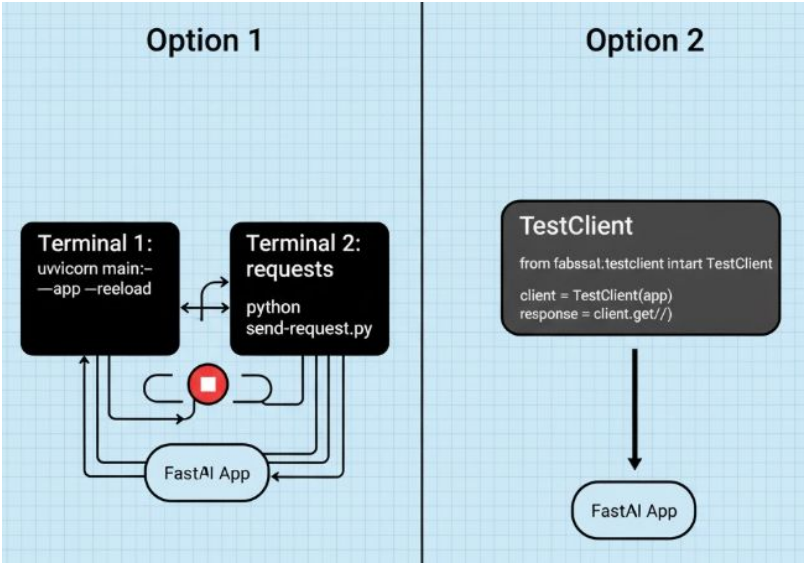
Pytest: Python's Simple, Powerful Testing Framework

- We will use Pytest to test our FastAPI application.
- How it works:
  - Run pytest in the terminal.
  - It auto-discovers files named test_*.py or *_test.py.
  - It runs any function inside those files named test_*().
  - It reports "Pass" or "Fail" for each test.



How to Test Our FastAPI API?

- **Option 1: The Hard Way**
  1. Start the unicorn server in one terminal.
  2. Write a separate Python script with requests to call it.
  3. Shut down the server.
  4. This is slow and complex to manage in a pipeline.
- **Option 2: The Easy Way**
  - FastAPI gives us a TestClient!



COLE

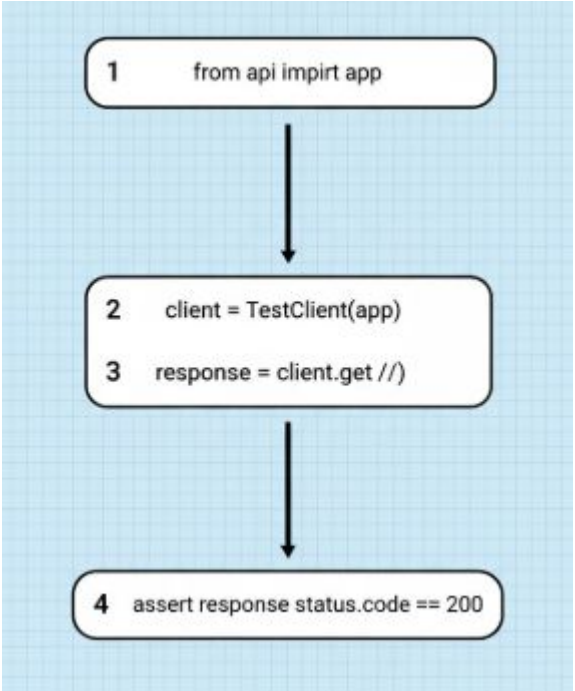
FastAPI's TestClient

- The TestClient lets us call our API routes *in Python* without needing a running server.
- It's fast, easy, and perfect for automated tests.
- # tests/test_api.py

  from fastapi.testclient import TestClient
  from api import app # Import our FastAPI app

  # Create a client that wraps our app
  client = TestClient(app)

  def test_read_main():
  # client.get() is just like calling the / route
  response = client.get("/")
  assert response.status_code == 200
  assert response.json() == {"message": "API is running"}



Example: A Health Check Test

- Let's create a new "health" endpoint (/) just for testing.
- **In [api.py](): **

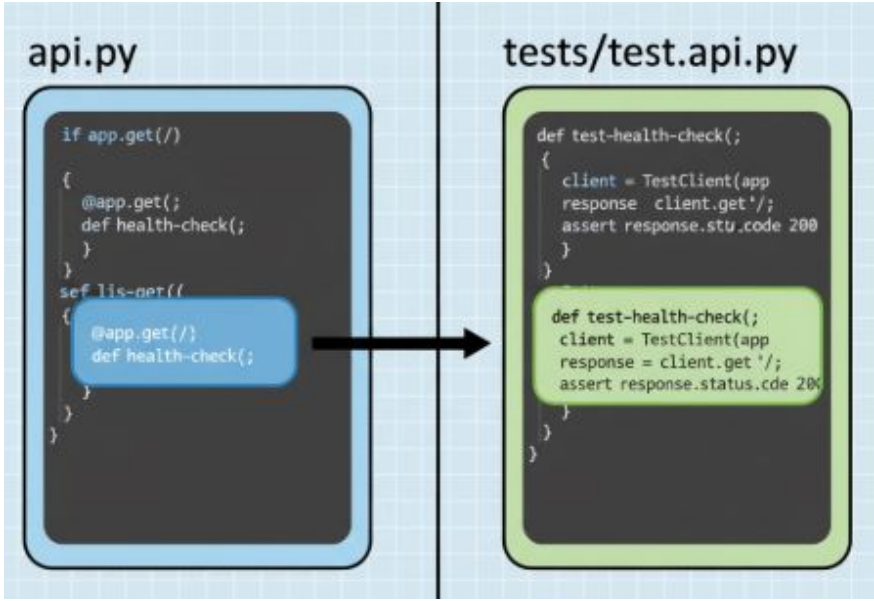
```
@app.get("/")
def read_root():
    return {"message": "API is running"}
```

- **In tests/test_api.py:**

```
from fastapi.testclient import TestClient
from api import app

client = TestClient(app)

def test_health_check():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "API is running"}
```



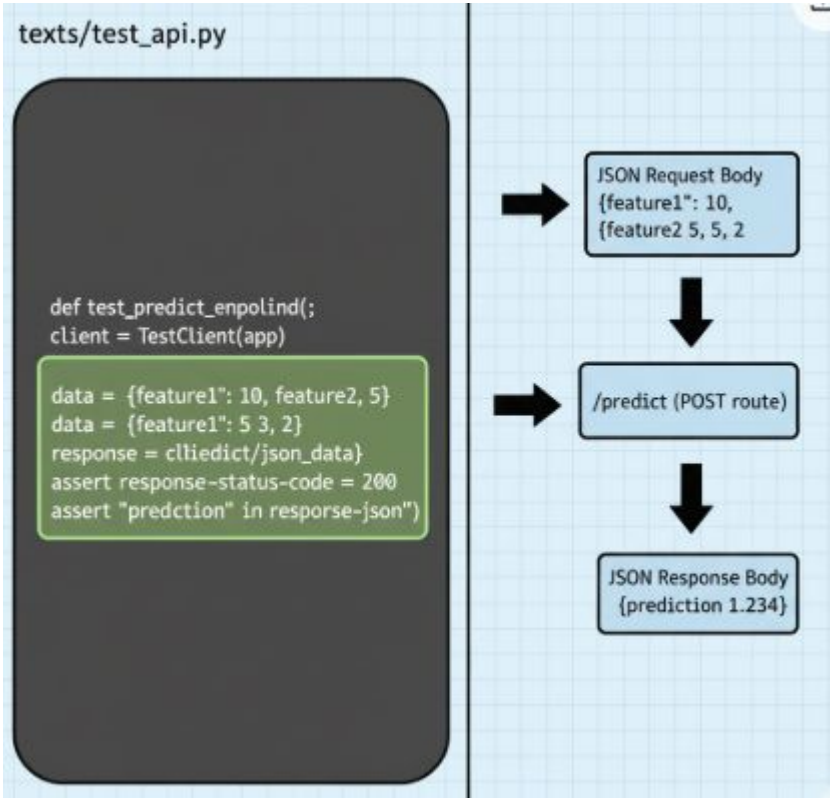
Example: A Prediction Test

- We can also test our /predict endpoint.
- **In tests/test_api.py:**

```
def test_predict_endpoint():
    # Send a valid JSON payload
    response = client.post(
        "/predict",
        json={"feature1": 10, "feature2": 5}
    )

    # Check that it succeeded
    assert response.status_code == 200

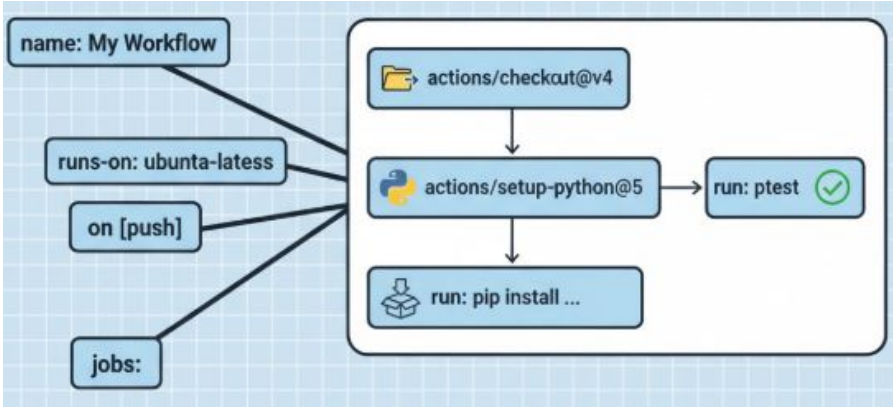
    # Check that the output format is correct
    data = response.json()
    assert "prediction" in data
```



Our Workflow: The Full Plan

Now we just tell GitHub Actions to do what we just did.

1. **Event:** On push...
2. **Job:** ...start a test job on an ubuntu-latest runner.
3. **Steps:**
   - actions/checkout@v4: Get our code.
   - actions/setup-python@v5: Install Python.
   - run: pip install ...: Install pytest, fastapi, etc. from requirements.txt.
   - run: pytest: Run our tests!



Our Workflow: main.yml

```yaml
# .github/workflows/main.yml
name: CI Pipeline
on: push
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: |
          pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run tests
        run: |
          pytest
```

COLE

Hands-on Lab

Your mission

1. **Prep:** Add pytest and fastapi[all] to requirements.txt.
2. **Test:** Create tests/test_api.py.
3. **Test:** Add the / health check endpoint to api.py.
4. **Test:** Write test_health_check() using TestClient.
5. **Automate:** Create .github/workflows/main.yml (from the previous slide).
6. **Run:** git add ., git commit, and git push.
7. **Observe:** Watch your pipeline run in the "Actions" tab on GitHub!

COLE

Q&A



Thank you for listening!

